Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №5

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: „Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии”

Выполнил:

Студент 3 курса

Группы ИИ-26

Петручик Д.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Вариант: 10

№ of variant	n	Task
10	7	AND

Ход работы:

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для своей логической функции (2^n примеров).
2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте np.random.seed(42) для воспроизводимости).
3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:

- Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
- Реализовать правильный градиент для BCE + сигмоида:

4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):

А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)

Б. MSE + адаптивный шаг (из ЛР №4)

В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)

Г. BCE + адаптивный шаг (по формуле из ЛР №2)

Критерий остановки для всех – суммарная ошибка Es ≤ Ee (возьмите Ee из ЛР №2 или №3, например 0.05 или 0.01).

5. Для каждой конфигурации:

- Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
- Зафиксировать количество эпох до достижения критерия остановки.
- После обучения посчитать accuracy на обучающей и тестовой выборках.
- Проверить, сколько процентов из всей таблицы истинности (2^n примеров) сеть предсказывает правильно.

6. Построить один общий график зависимости суммарной ошибки Es от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).

7. Реализовать режим функционирования сети:

- Пользователь вводит вектор из n значений (0 или 1).
- Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
- Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».

8. Написать вывод (обязательно ответить на следующие пункты):

- Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
- Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
- Как изменилась обобщающая способность (accuracy на тесте и % восстановления полной таблицы) при переходе на BCE?
- Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?
- Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Допускается: NumPy, Matplotlib

Запрещено: любые ML-фреймворки (scikit-learn, PyTorch, TensorFlow и т.д.)

Структура отчёта:

1. Условие лабораторной работы
2. Код программы (с комментариями к изменениям ВСЕ)
3. Таблица результатов (количество эпох, accuracy train/test, % правильных предсказаний на всей таблице истинности)
4. График сходимости (все 4 кривые)
5. Вывод (ответы на ВСЕ вопросы)

Код программы:

```
import numpy as np

import matplotlib.pyplot as plt

import itertools


num_inputs = 7

stop_error_threshold = 0.01

fixed_learning_rate = 0.1

max_epochs = 10000


np.random.seed(42)


full_input_matrix = np.array(

    list(itertools.product([0, 1], repeat=num_inputs)),

    dtype=float

)


full_output_vector = np.array(

    [1 if np.all(sample == 1) else 0 for sample in full_input_matrix],
```

```
    dtype=float
)

indices = np.random.permutation(len(full_input_matrix))
train_size = int(0.8 * len(full_input_matrix))

train_indices = indices[:train_size]
test_indices = indices[train_size:]

train_inputs = full_input_matrix[train_indices]
train_outputs = full_output_vector[train_indices]

test_inputs = full_input_matrix[test_indices]
test_outputs = full_output_vector[test_indices]

train_inputs_with_bias = np.c_[np.ones(len(train_inputs)), train_inputs]
test_inputs_with_bias = np.c_[np.ones(len(test_inputs)), test_inputs]
full_inputs_with_bias = np.c_[np.ones(len(full_input_matrix)), full_input_matrix]

def sigmoid_activation(linear_output):
    linear_output = np.clip(linear_output, -100, 100)
    return 1 / (1 + np.exp(-linear_output))

def train_perceptron_model(input_matrix, target_outputs,
                           loss_function_type, learning_rate_mode):
    np.random.seed(42)
    weight_vector = np.random.uniform(-0.1, 0.1, num_inputs + 1)
    error_history_per_epoch = []
```

```
for epoch_idx in range(max_epochs):
    total_epoch_error = 0.0

    for sample_idx in range(len(input_matrix)):
        current_input = input_matrix[sample_idx]
        desired_output = target_outputs[sample_idx]

        predicted_output = sigmoid_activation(
            np.dot(weight_vector, current_input)
        )

        if learning_rate_mode == "fixed":
            current_learning_rate = fixed_learning_rate
        else:
            current_learning_rate = 1.0 / (
                1.0 + np.sum(current_input ** 2)
            )

        if loss_function_type == "MSE":
            total_epoch_error += 0.5 * (
                desired_output - predicted_output
            ) ** 2

            gradient = (
                (desired_output - predicted_output)
                * predicted_output
                * (1 - predicted_output)
            )
```

```
        elif loss_function_type == "BCE":
            epsilon = 1e-15
            clipped_prediction = np.clip(
                predicted_output,
                epsilon,
                1 - epsilon
            )

            total_epoch_error += -(
                desired_output * np.log(clipped_prediction)
                + (1 - desired_output)
                * np.log(1 - clipped_prediction)
            )

            gradient = desired_output - predicted_output

        else:
            raise ValueError("error")

        weight_vector += (
            current_learning_rate * gradient * current_input
        )

    error_history_per_epoch.append(total_epoch_error)

    if total_epoch_error <= stop_error_threshold:
        break

return weight_vector, error_history_per_epoch, epoch_idx + 1
```

```
training_configs = {
    "A. MSE + fixed": ("MSE", "fixed"),
    "B. MSE + adaptive": ("MSE", "adaptive"),
    "C. BCE + fixed": ("BCE", "fixed"),
    "D. BCE + adaptive": ("BCE", "adaptive")
}

training_results = {}

for config_name, (loss_func, lr_mode) in training_configs.items():
    training_results[config_name] = train_perceptron_model(
        train_inputs_with_bias,
        train_outputs,
        loss_func,
        lr_mode
    )
    print(f"{config_name} completed. Epochs: {training_results[config_name][2]}")

plt.figure(figsize=(10, 6))

curve_colors = ["blue", "green", "red", "purple"]
line_styles = ["-", "--", "-.", ":"]

for (config_name, (final_weights, error_history, total_epochs)), color, style in zip(
    training_results.items(),
    curve_colors,
    line_styles
):
```

```
    plt.plot(
        error_history,
        label=f"{config_name} ({total_epochs} ep)",
        color=color,
        linestyle=style
    )

plt.axhline(
    y=stop_error_threshold,
    color="orange",
    linestyle="--",
    label=f"Ee = {stop_error_threshold}"
)

plt.yscale("log")
plt.title("Convergence")
plt.xlabel("Epochs")
plt.ylabel("Error")
plt.legend()
plt.grid(True)
plt.show()

def compute_accuracy(model_weights, input_matrix, true_outputs):
    probabilities = sigmoid_activation(np.dot(input_matrix, model_weights))
    predictions = (probabilities >= 0.5).astype(int)
    return np.mean(predictions == true_outputs) * 100

print("\nResults:")
for config_name, (final_weights, error_history, total_epochs) in training_results.items():
```

```
    train_accuracy = compute_accuracy(
        final_weights,
        train_inputs_with_bias,
        train_outputs
    )

    test_accuracy = compute_accuracy(
        final_weights,
        test_inputs_with_bias,
        test_outputs
    )

    full_accuracy = compute_accuracy(
        final_weights,
        full_inputs_with_bias,
        full_output_vector
    )

    print(
        f"{config_name:<15} | Ep: {total_epochs:<5} | "
        f"Train: {train_accuracy:>5.1f}% | "
        f"Test: {test_accuracy:>5.1f}% | "
        f"Full: {full_accuracy:>5.1f}%"
    )

print("\nOperation mode:")
final_model_weights = training_results["D. BCE + adaptive"][0]

while True:
```

```
user_input_str = input("Enter 7 bits or q: ")

if user_input_str.lower() == "q":
    break

try:
    user_input_vector = np.array(
        [int(bit) for bit in user_input_str.split()],
        dtype=float
    )

    if len(user_input_vector) != num_inputs:
        print("error")
        continue

    user_input_with_bias = np.insert(user_input_vector, 0, 1)

    output_probability = sigmoid_activation(
        np.dot(final_model_weights, user_input_with_bias)
    )

    predicted_class = 1 if output_probability >= 0.5 else 0
    true_class = 1 if np.all(user_input_vector == 1) else 0

    print(f"ŷ: {output_probability:.4f}")
    print(f"class: {predicted_class}")
    print("ok" if predicted_class == true_class else "error")

except:
```

```
    print("error")
```

Результат программы:

```
A. MSE + fixed completed. Epochs: 212
B. MSE + adaptive completed. Epochs: 71
C. BCE + fixed completed. Epochs: 806
D. BCE + adaptive completed. Epochs: 253

Results:
A. MSE + fixed  | Ep: 212   | Train: 100.0% | Test:  96.2% | Full:  99.2%
B. MSE + adaptive | Ep: 71    | Train: 100.0% | Test:  96.2% | Full:  99.2%
C. BCE + fixed  | Ep: 806   | Train: 100.0% | Test:  96.2% | Full:  99.2%
D. BCE + adaptive | Ep: 253   | Train: 100.0% | Test:  96.2% | Full:  99.2%

Operation mode:
Enter 7 bits or q: 1 1 1 1 1 1 1
ŷ: 0.0000
class: 0
error
Enter 7 bits or q: 0 0 0 0 0 0 0
ŷ: 0.0027
class: 0
ok
Enter 7 bits or q:
```

Вывод: Проделав данную лабораторную работу реализовали однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных.